Routines Page — Information Document

Page Overview

Purpose

The Routines page is the management surface for scheduled, automatic sweep jobs. A routine bundles a saved sweep configuration (repositories, keywords, optional date range, max-results cap) with a 5-field cron expression, plus per-routine flags for email, AI summarization, and desktop completion notifications. When the schedule fires, the app runs an `automated_sweep` execution against the configured repositories, stores the report, and (optionally) emails it and/or raises a desktop notification.

Routines may be pinned to either of two execution locations: **Local** (fired by the APScheduler background thread inside the local `resmon` daemon) or **Cloud** (fired by the `resmon-cloud` scheduler, via the JWT-gated `/api/v2/routines` service). Both kinds are rendered side-by-side in a single unified table.

Primary User Flows

1. **Create a routine.** Open the Create modal, optionally load a saved routine configuration via `ConfigLoader`, pick repositories, keywords, an optional date range, a max-results cap, flags (AI / Email / Results-in-Email / Notify-on-Completion), a cron expression, and an execution location. Submit to persist.
2. **Edit an existing routine.** Click **Edit** on a local row. The modal pre-populates from the existing row; saving issues `PUT /api/routines/{id}`.
3. **Activate / deactivate.** Toggle scheduling without deleting via the Activate / Deactivate action button. Deactivation removes the APScheduler job but preserves the DB row.
4. **Per-row quick toggles.** The Email, AI, and Notify columns expose single-click toggles that patch the matching flag with `PUT /api/routines/{id}`.
5. **Migrate between locations. Move to Cloud** and **Move to Local** trigger a confirmation modal, then perform a two-step destination-first create / source-delete migration.
6. **Cancel a live run.** When a routine is currently firing, a **Cancel Run** button appears on its row and calls through `ExecutionContext.cancelExecution`.

7. **Delete a routine.** Delete removes the DB row (local) or the cloud record; historical execution rows produced by that routine are retained.
8. **Materialized via Configurations import.** Importing a routine config on the Configurations page automatically materializes a matching local routine row on this page (`is_active=False` by default, so the imported routine is deactivated until the user explicitly activates it). The page subscribes to the configurations bus (`useConfigurationsVersion()`) so newly-imported routines surface here without a manual reload. This keeps the Routines list and the routine-configs list in lockstep — every routine config has a real routine, and vice-versa.

Inputs and Outputs

Inputs:

- Routine name, cron expression, repositories, optional date range, keywords, max-results cap.
- Flags: `is_active` , `email_enabled` , `email_ai_summary_enabled` , `ai_enabled` , `notify_on_complete` , `execution_location` .
- Optional `ai_settings` and `storage_settings` payloads (stored as JSON; not edited directly on this page).

Outputs:

- Persisted `routines` rows (local) or `routines` documents on the cloud scheduler.
- APScheduler jobs registered under the routine's id as `job_id` .
- Automated `executions` rows produced at each fire, stamped with `routine_id` and `last_executed_at` .
- Downstream report artifacts (Markdown / PDF / figures) under `resmon_reports/` .

Known Constraints or Permissions

- Cloud rows render only when the user is signed in (`useAuth().isSignedIn`); cloud list failures are swallowed and the table simply shows the local half.
- The **Execution location** → **Cloud** radio is disabled unless cloud sync is enabled (signed in).
- Cron is 5-field (`m h dom mon dow`); the default value in the create form is `0 8 * * *` .
- Only local rows expose the Edit, Email/AI/Notify toggles, Activate/Deactivate, Last Execution, and Last Status columns — cloud rows show em-dashes in those columns because cloud state is not mirrored into the local DB for rendering.
- Migration atomicity is orchestrated renderer-side: destination is created first; source is deleted only if that succeeded.

Frontend

Route and Main Component

- Route: `/routines`.
- Main component: `resmon_scripts/frontend/src/pages/RoutinesPage.tsx`.

Child Components and Hooks

Child components:

- `RoutineEditModal` (from `components/Routines/RoutineEditModal.tsx`) — the create/edit modal has been extracted into a reusable component shared with the Calendar page (whose event popover exposes an `Edit Routine` button). On save it broadcasts BOTH `notifyRoutinesChanged()` (via `lib/routinesBus.ts`) and `notifyConfigurationsChanged()` (via `lib/configurationsBus.ts`) so every routine-aware surface refetches without a manual reload. The host pages own only `editTarget: Routine | null` and an `editOpen` flag; all form state, validation, and the inner editor children below live inside the modal.
- `RepositorySelector` (multi-select repository picker).
- `KeywordCombinationBanner` — mounted in the create/edit modal under the selector; lists the upstream keyword-combination semantics for each selected repository, with a tooltip pointing to the consolidated glossary on the Repositories & API Keys page.
- `DateRangePicker`, `KeywordInput`, `ScheduleConfigurator` (cron builder).
- `ConfigLoader` with `configType="routine"` — populates everything except the date range, which is re-picked per run.
- `RepoKeyStatus` with `variant="routine"` — one row per selected repository, showing credential presence.
- `PageHelp` and `InfoTooltip` — help UI only.

Hooks:

- `useAuth()` — source of `isSignedIn`, gating cloud rows and the Cloud execution-location radio.
- `useExecution()` — supplies `activeExecutions`, `cancelExecution`, and `completionCounter`. `completionCounter` is a dependency of both fetchers so the table refreshes when an execution finishes.
- `useRepoCatalog()` — `bySlug` / `presence` / `refreshPresence` back the per-row `RepoKeyStatus` entries.

- `useRoutinesVersion()` (from `lib/routinesBus.ts`) — dependency of the local and cloud fetch effects so saves originating from the Calendar page's `Edit Routine` modal (or any other future surface) refresh the table here.

API clients:

- `apiClient` for the local `/api/routines` surface.
- `cloudClient` for the cloud `/api/v2/routines` surface.

UI State Model

Top-level state:

- `routines: Routine[]` — local rows from `GET /api/routines`.
- `cloudRoutines: CloudRoutineRow[]` — cloud rows from `GET /api/v2/routines`.
- `loading: boolean`, `error: string`.
- `pendingMigration: PendingMigration` — `{ kind: 'to-cloud' | 'to-local', routine: UnifiedRoutine } | null`; drives the confirm modal.
- `migrating: boolean` — disables the confirm modal's buttons while the two-step migration runs.

Create/edit form state:

- `formOpen`, `editId` (`null` = creating, number = editing a local id).
- `formName`, `formCron` (default `"0 8 * * *"`), `formRepos`, `formDateFrom`, `formDateTo`, `formKeywords`, `formMaxResults` (default `100`).
- `formAi`, `formEmail`, `formEmailAi`, `formNotify`, `formLocation` (`'local' | 'cloud'`).

Types:

- `Routine` — local row shape.
- `CloudRoutineRow` — `{ routine_id, name, cron, enabled, parameters }` as returned by `/api/v2/routines`.
- `UnifiedRoutine` extends `Routine` with `execution_location` and optional `cloud_id`. It is the shape handed to the migration modal so both row types share a single confirm path.

Helpers:

- `lastStatusBadgeClass(status)` maps status → badge class: `completed → badge-success`, `failed → badge-error`, `cancelled → badge-cancelled`, everything else (`running`,

`cancelling` , `scheduled` , `unknown`) → `badge-info` . This mirrors the Dashboard and Results & Logs palette.

Key Interactions and Events

- **Fetch.** `fetchRoutines` → `GET /api/routines` ; `fetchCloudRoutines` → `GET /api/v2/routines` (no-op when `cloudSyncEnabled` is false). Both run on mount and every time `completionCounter` changes.
- **Create.** `handleSubmit` when `editId === null` :
  - If `formLocation === 'cloud'` and signed in: `POST /api/v2/routines` with `{ name, cron, parameters, enabled: true }` .
  - Otherwise: `POST /api/routines` with the full local body (flags + `execution_location: 'local'`).
- **Edit.** `handleSubmit` when `editId !== null` : `PUT /api/routines/{editId}` with name, cron, parameters, `is_active: true` , all flag fields, and `execution_location` .
- **Delete.** `handleDelete(id)` → `DELETE /api/routines/{id}` (local); `handleDeleteCloud(cloudId)` → `DELETE /api/v2/routines/{cloudId}` (cloud).
- **Toggle active.** `handleToggleActive` calls `POST /api/routines/{id}/activate` or `POST /api/routines/{id}/deactivate` based on current `is_active` .
- **Quick toggles.** `handleToggleEmail` , `handleToggleAi` , `handleToggleNotify` each call `PUT /api/routines/{id}` with only the inverted flag in the body.
- **Move to Cloud.** `performMoveToCloud` : `POST /api/v2/routines` (cloud create), then `POST /api/routines/{id}/released-to-cloud` (local delete).
- **Move to Local.** `performMoveToLocal` : `POST /api/routines/adopt-from-cloud` (local create), then `DELETE /api/v2/routines/{cloud_id}` (cloud delete).
- **Cancel running fire.** For each local row, the render scans `activeExecutions` for `executionType === 'automated_sweep'` with matching `routine_id` and status `running` or `cancelling` ; if present, a **Cancel Run** button calls `cancelExecution(running.executionId)` and shows a `Stopping…` state while `cancelling` .

Error and Empty States

- **Loading.** `if (loading)` returns a muted "Loading routines…" placeholder.
- **Empty.** When both `routines` and `cloudRoutines` are empty, the table body shows a single "No routines configured." row spanning all 10 columns.
- **Errors.** A non-empty `error` string renders a `form-error` banner above the table. Cloud fetch errors are swallowed intentionally (sign-in dialogs / toasts elsewhere handle auth failures).

- **Missing cloud_id on move-to-local.** `performMoveToLocal` sets `"Missing cloud_id for migration"` and returns without calling any endpoint.
- **Partial migration failures.** If the destination create succeeds but the source delete fails, the error is surfaced and both rows remain; retrying the delete on the second attempt recovers without losing the destination copy (per the renderer-side atomicity comment in the component).

Backend

API Endpoints

Local (`resmon_scripts/resmon.py`):

- `GET /api/routines` — list, enriched with `last_execution` (from `last_executed_at`) and `last_status` (most recent `executions` row by `routine_id`).
- `POST /api/routines` — create (body: `RoutineCreate`).
- `PUT /api/routines/{routine_id}` — partial update (body: `RoutineUpdate` , every field optional).
- `DELETE /api/routines/{routine_id}` — delete.
- `POST /api/routines/{routine_id}/activate` — set `is_active=1` .
- `POST /api/routines/{routine_id}/deactivate` — set `is_active=0` .
- `POST /api/routines/{routine_id}/released-to-cloud` — delete the local row after a successful cloud mirror.
- `POST /api/routines/adopt-from-cloud` — insert a local row populated from a cloud body (preserves name, cron, parameters verbatim).

Cloud (`resmon-cloud` , consumed via `cloudClient`):

- `GET /api/v2/routines` — list cloud routines (`CloudRoutineRow[]`).
- `POST /api/v2/routines` — create a cloud routine (`{ name, cron, parameters, enabled }`).
- `DELETE /api/v2/routines/{routine_id}` — delete a cloud routine.

Request and Response Patterns

- Local request bodies follow the Pydantic models `RoutineCreate` and `RoutineUpdate` . `RoutineCreate` requires `name` , `schedule_cron` , `parameters` ; defaults `is_active=True` , all flags `False` , and `execution_location="local"` . `RoutineUpdate` makes every field optional and only applies the keys the renderer actually sends.
- `execution_location` is validated to be `'local'` or `'cloud'` ; anything else returns `400` .
- Create returns `{id, name}` with status `201` .

- Update returns `{id, ...updates}`.
- `released-to-cloud` returns `{released: True, id}`; `adopt-from-cloud` returns `{id, name, execution_location: "local"}` with status `201`.
- Cloud requests use the dedicated `cloudClient` wrapper so JWT headers (IMPL-30) are applied; cloud bodies use the short `cron` / `enabled` / `routine_id` field set.

Persistence Touchpoints

- SQLite `routines` table is the authoritative store for local rows (written via `insert_routine`, `update_routine`, `delete_routine`, `get_routines`, `get_routine_by_id`).
- `parameters`, `ai_settings`, and `storage_settings` are serialized as JSON strings. Integer-encoded booleans are used for `is_active`, `email_enabled`, `email_ai_summary_enabled`, `ai_enabled`, and `notify_on_complete`.
- `executions` rows produced by a routine fire carry the FK `routine_id`; the list endpoint joins the most recent such row to expose `last_status`.
- APScheduler job state lives in a separate SQLAlchemy job store (SQLite) managed by `ResmonScheduler`; job id is the string form of the routine id.
- `_sync_routine_config` / `_delete_routine_config` keep the per-routine configuration mirror in sync on every mutation (IMPL-R5).

Execution Side Effects

Scheduler ↔ CRUD sync (IMPL-R5). Each mutating endpoint calls a helper that mirrors the change into APScheduler:

- `_sched_add_routine(id)` — registers the job if the row is active and `execution_location != 'cloud'`.
- `_sched_update_routine(id)` — re-adds an active local job, otherwise removes it.
- `_sched_remove_routine(id)` — removes the job unconditionally.

All three swallow scheduler exceptions (logged, not propagated) so a jobstore fault cannot abort the underlying DB mutation.

Startup reconciliation (IMPL-R4). The `_init_scheduler_on_startup` startup hook:

1. Installs `_dispatch_routine_fire` via `set_dispatcher`.
2. Instantiates `ResmonScheduler` (with a disposable on-disk jobstore when the app DB is `:memory:` under tests).
3. Starts the scheduler.

4. Walks `get_routines(conn)` and calls `scheduler.add_routine(r)` for every row with `is_active` truthy; failures are logged per-row.

Routine fire pipeline (`_dispatch_routine_fire` , IMPL-R6). On every scheduled fire APScheduler calls `_routine_callback` , which delegates to the installed dispatcher. The dispatcher:

1. Loads the routine row; exits if missing or inactive.
2. Parses `parameters` JSON (unparseable payloads are logged and dropped).
3. Calls `admission.try_admit(kind="routine", routine_id=..., params_json=...)` ; returns early if the admission controller enqueues or drops the fire.
4. Loads `ai_settings` JSON, constructs a `SweepEngine` with `ai_enabled` and `ai_settings` , and calls `engine.prepare_execution("automated_sweep", repositories, params)` to reserve the execution row.
5. Stamps the `routine_id` column on the new `executions` row.
6. Registers the execution with `progress_store.register(exec_id)` and calls `_launch_execution(engine, exec_id, conn, ephemeral_credentials=None)` to run the sweep (the admission slot is released inside `_launch_execution` 's `finally` via `admission.note_finished`).
7. Updates `routines.last_executed_at = datetime('now')` .

APScheduler job options. Jobs are added with `replace_existing=True` , `coalesce=True` , and `misfire_grace_time=60` (see `ResmonScheduler.add_routine`), so overlapping missed fires collapse into one and late fires within 60 seconds still run.

Downstream effects of a fire. Depending on the routine's flags, a successful fire may additionally: write report artifacts under `resmon_reports/` , send an email via the SMTP pipeline (when `email_enabled` ; `email_ai_summary_enabled` inlines the AI summary), and raise a desktop notification (when `notify_on_complete` is ON and the global Settings → Notifications policy permits it).

Migration (IMPL-37). Local ⇄ cloud migration is orchestrated by the renderer in two steps; the two local endpoints (`released-to-cloud` , `adopt-from-cloud`) own only their half. Historical execution rows stay attached to the side that produced them.

Cloud path. Cloud routines are not registered with the local APScheduler; the cloud scheduler (`resmon-cloud` , per IMPL-32) owns their lifecycle. The desktop only reads and writes them through `cloudClient` .

Routines Page — Information Document

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints or Permissions

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Backend

API Endpoints

Request and Response Patterns

Persistence Touchpoints

Execution Side Effects